

Discrete Optimization Specialization: Assignment 9

Birthday Parade

1 Problem Statement

The Dowager Empress is treated to a military parade by the finest soldiers in the empire, her personal guard. The soldiers are lined up columns with the captain in front, and parade in front of her one after the other, starting with the captain. Each soldier can only parade in front of her when all soldiers ahead of that soldier in their column have already paraded.

The Dowager Empress would like to believe that all her soldiers are identical, so she remains pleased during the parade until she the height difference between two consecutive soldiers is discernable to her. The aim is to keep the Dowager Empress pleased for as long as possible.

2 Data Format Specification

The input form for the Birthday Parade is a file named `data/paradep.dzn`, where p is the problem number, n is the number of soldiers in her personal guard, *height* is an array of heights for each soldier, *cols* is the number of columns they are arrayed in, *delta* is the height difference which is indiscerable with the Dowager Empress's failing eyesight.

The captain is always soldier one, soldier 2 is the first soldier in the first column, soldier $cols + 1$ is the first soldier in the last column, soldier $cols + 2$ is the second soldier in the first column, etc.

The data declarations and main decisions are hence:

```
int: n;                                % number of soldiers
set of int: SOLDIER = 1..n;
set of int: POS = 1..n;
array[SOLDIER] of int: height; % height of each soldier
int: cols;                            % number of columns of soldiers
int: delta;                           % height delta allowed

array[SOLDIER] of var POS: pos; % position of each soldier
```

The output required for every stage is of the form

```
pos = [position of each soldier in the parade];
happiness = how long the Dowager Empress is happy;
```

Note the Dowager Empress is happy until the first position in the list where the soldier is noticeably different in height (difference is greater than *delta*) than the previous soldier. If this never occurs she is happy for the entire parade (*happiness* = n).

The aim of this assignment is to model the problem, and give a basic search strategy (just one `int_search` annotation) that will keep the Dowager Empress happy as long as possible.

For example given the data file

```

n = 12;
height = [4, 3, 3, 5, 2, 5, 3, 4, 2, 1, 2, 4];
cols = 4;
delta = 1;

```

the soldier are arranged in 4 columns behind the captain, as

```

      1
    2  3  4  5
    6  7  8  9
   10 11 12

```

An optimal solution is given by

```

happiness = 11;
pos = [1, 9, 4, 2, 5, 11, 6, 3, 7, 12, 8, 10];

```

Note e.g. that soldier 11 is paraded 8th after both the soldiers in front of them in the column, that is soldier 3 (paraded 4th) and soldier 7 (paraded 6th) are already paraded. Note that the heights of the soldiers in the order paraded are 4, 5, 4, 3, 2, 3, 2, 2, 3, 4, 5, 1 and the Emperor Dowager is happy until the last soldier parades.

3 Instructions

Edit `parade.mzn` to solve the optimization problem described above. Your `parade.mzn` implementation can be tested on the data files provided. In the MINIZINC IDE, use the *Run* icon to test your model locally. At the command line use,

```
mzn-gecode ./parade.mzn ./data/<inputFileName>
```

to test locally. In both cases, your model is compiled with MINIZINC and then solved with the GECODE solver.

Resources You will find several problem instances in the `data` directory provided with the hand-out.

Handin This assignment contains XX solution submissions and YY model submissions. For solution submissions, we will retrieve the best/last solution the solver has found using your model and check its correctness and quality. For model submissions, we will retrieve your model file (`.mzn`) and run it on some hidden data to perform further tests.

From the MINIZINC IDE, the *Submit to Coursera* icon can be used to submit assignment for grading. From the command line, `submit.py` is used for submission. In both cases, follow the instructions to apply your MINIZINC model(s) on the various assignment parts. You can submit multiple times and your grade will be the best of all submissions.¹ It may take several minutes before your assignment is graded; please be patient. You can track the status of your submission on the *programming assignments* section of the course website.

¹Solution submissions can be graded an unlimited number of times. However, there is a limit on grading of **model submissions**.

4 Technical Requirements

For completing the assignment you will need MINIZINC 2.1.x and the GECODE 5.0.x solver. Both of these are included in the bundled version of the MINIZINC IDE 2.1.x (<http://www.minizinc.org>). To submit the assignment from the command line, you will need to have Python 3.5.x installed.